Question: Pick from both sides

Given an array of N elements, pick B elements from the corners to maximise the sum.

B = 3

A : 5 -2 3 1 2
0 1 2 3 4

5 mins!

3 + 1 + 2 = 6

5 + 2 + 1 = $\boxed{8}$

Approach1 : X

Sorting will not work

Approach2 : X

B = 3

A = 5 -2 3 1 2
0 1 2 3 4

L R

5 + 2 + 1 = $\boxed{8}$

A = -2 100 200 1 2 3

B = 3

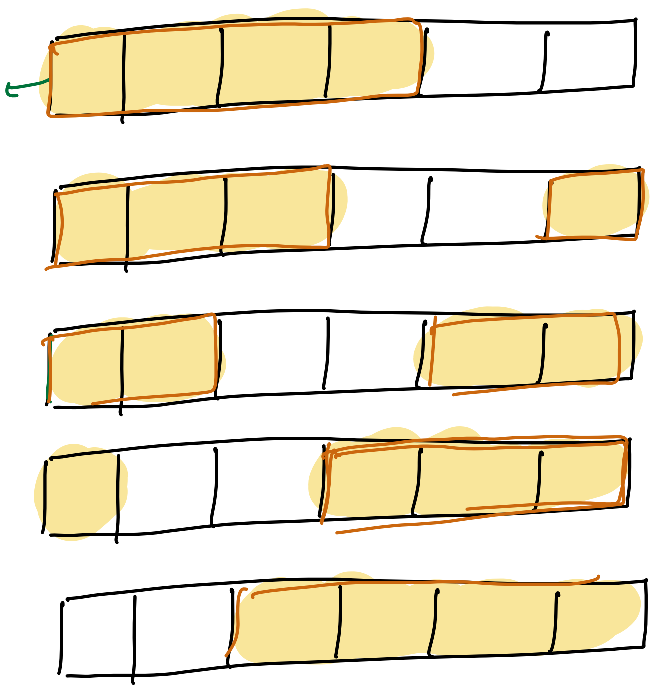
$3 + 2 + 1 = \boxed{6}$

$-2 + 100 + 200 = 298$

N = 6

B = 4

Solution :



→ 0

→ 1

→ 2

→ 3

⋮

B

[0, 1, 2, ..., B]

$\boxed{B+1}$

Brute Force

possibility = B + 1

iterations per possibility = B

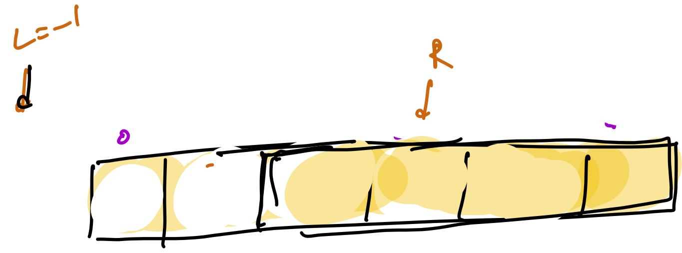
T.C : $O((B+1)\cdot B) = O(B^2)$

Approach2:

$sum(s \dots e) = ps[e] - ps[s-1]$ ✓

$Sum(0 \dots e) = ps[e]$

$L = B-1$
$R = N$
$B = 4$



pre[-1]

ans =

case1:

Left : $ps[L]$

Right : $e = N-1, \quad s = N \Rightarrow$

$pre[e] - pre[s-1]$

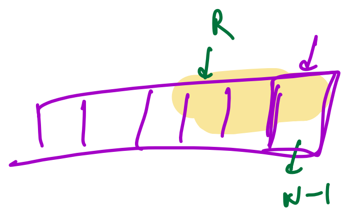
$pre[N-1] - pre[N-1] = 0$

Case 2:

Left = $pre[L]$

Right = $pre[N-1] - pre[R-1]$

$= pre[N-1] - pre[N-2]$




```
int ans = -INF;
L = B-1          R = N

while( L >= 0 ) {
    sumPossibility = pre[L] + pre[N-1] - pre[R-1]
                     (left)   (right)
    ans = max(ans, sumPossibility);
    L--;
    R--;
}
```

$L = 0$

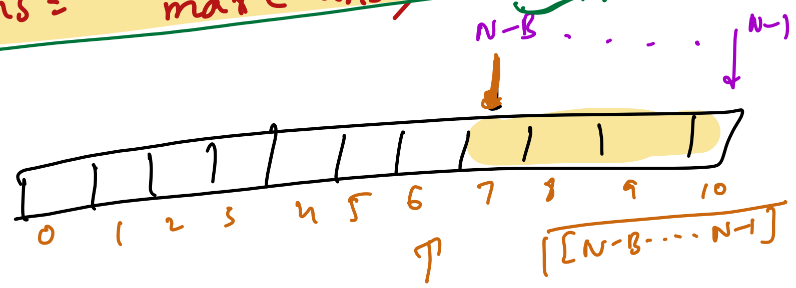
left = $pre[0]$ = { first ele }

right = $pre[N-1] - \dots$ { B-1 elements }

$[N-B \dots N-1]$

$pre[e] - pre[s-1]$

$\boxed{ans = \max(ans,\ pre[N-1] - pre[N-B-1])}$



B = 4

B = 5

N-B

[N-B...., N-1]

$(N-1) - (N-B)+1$

$\cancel{N}-\cancel{1}-\cancel{N}+B+\cancel{1} = \boxed{B}$

T.C:

1) construct prefix sum : O(N)

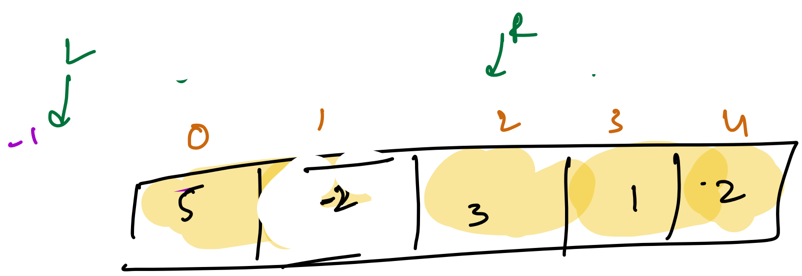
2) consider all possibility : O(B)

Total T.C : O(N+B) ⇒ O(N)

S.C: O(N)

↓

prefix sum array



N=5

B=3

$\boxed{ans : \cancel{6}\ 8}$

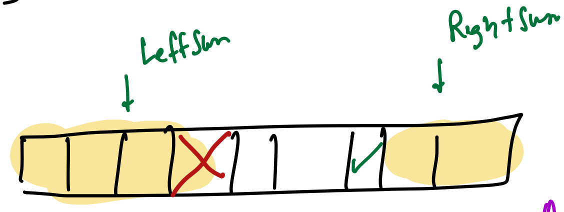
Case1 : : 6 + 0

Case2 : : 3 + 2 = 5

Case3 : 5 + 3 = 8

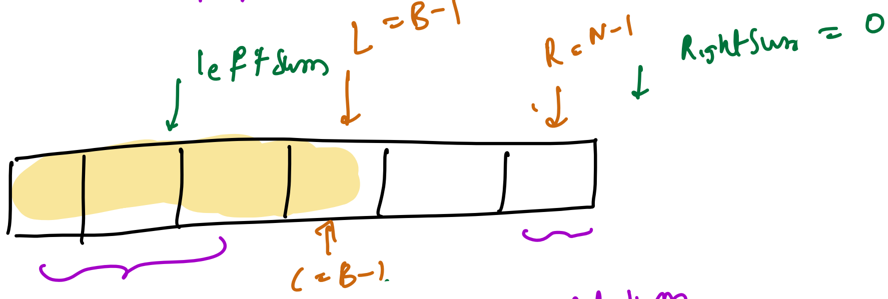
Case : 0 + 6 = 6

Approach 3 :



Leftsum = Leftsum − A[i]

RightSum = Rightsum + A[j]



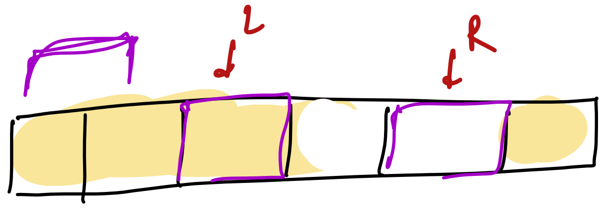
Case1: leftsum + Right sum

Case2 : leftsum = leftsum − A[B-1]

RightSum = Rightsum +

lSum = sum(B)

rSum = 0



Pseudocode (Lecture Notes)

Case1: lSum + rSum

Case2: lsum = lsum − A[L]

rSum = rsum + A[R]

ans = max(ans, lsum + rsum);

5 min

Question: Single Number 3

All elements except 2 elements occur twice. Find these elements which occur once

XOR => 1 ^ 3

A = 1 2 5 2 5 4 3 4

Ans = [1, 3]

(10, 6)

$\boxed{6, 10}$

Approach1: Hashmaps.

→ Store frequency of every element
→ Iter over hashmap → return element with frequency

T.C: O(N)
S.C: O(N) → Hashmap

Approach2:
Sort the array

T.C: O(NlogN)
S.C: O(1)

sort(A) 1 2 2 3 4 4 5 5

ans = 1 + 1 = $\boxed{2}$ => $\boxed{O(N)}$

Approachs :

XOR(array) = $1 \wedge 3$ $\quad (a \wedge b)$

$$\begin{array}{rccc} & 0 & 0 & 1 \\ \wedge & 0 & 1 & 1 \\ \hline & 0 & 1 & 0 \end{array} \quad \boxed{2}$$

$a \wedge b = 0\ 1\ 0$

a = _ _ 1(0) 0(1)

b = _ _ 0(1) 0(1)

0 ⇒ 1 will feed

1 ⇒ 0 will feed

We cannot find a, b if given $a \wedge b$

$a \wedge b \wedge c$ ⇒ we cannot find a, b, c

$a \wedge b = 0\ 1\ 0$ (i)

a = _ 1 _

b = _ 0 _

$\boxed{j = 1}$

$x \wedge y = 1$

$0 \wedge 1 = 1 \wedge 0$

If ith bit of $a \wedge b$ is 1, then a and b will have different bits at ith position

1st Part : ith bit is 1 ✓

2nd Part : ith bit is 0 ✓

a, b will be in different parts

$\boxed{32 \text{ bit}}$

0 0 0 0 0 0 0 0

A = 1 2 5 2 5 4 3 4

001 010 101 010 101 100 011 100

XOR(array) = $a \wedge b$ = $1 \wedge 3$ = 0 1 0

1st bit

$0 \wedge 0 = 0$

$1 \wedge 1 = 0$

Arr1 = [2, 2, 3] → XOR(Arr1) = 3

Arr2 = [1, 5, 5, 4, 4] → XOR(Arr2) = 1

Steps:

O(N) ← 1) Find XOR of all elements = x_r

log(n) ← 2) Find index of any set in x_r

x_r = 0 1 1 0 1 0 1 0 0 0 0

10 9 8 7 6 5 4 3 2 1 0

i = ~~0~~ ~~1~~ ~~2~~ ~~3~~ 4

i = 4

$\boxed{i = 0;}$ → $\boxed{O(1)}$

```
while ( xr > 0 ){
    if( xr & 1 == 1 )
        index = i;
        break;
    }
    i++;
    xr = xr >> 1;
}
```

O(logN)

$\boxed{i = 1}$

$\underbrace{x_r \rightarrow \frac{x_r}{2} \rightarrow \frac{x_r}{4} \rightarrow \frac{x_r}{8} \ldots\ldots 0}_{\log(x_r)}$

1 2 5 2 5 4 3 4

001 010 101 010 101 100 011 100

010 => 1<<1

Step1: $\boxed{X_r} = 1 \wedge 3 =$ X_r: 0 1 0

2 1 0

A[i] & 010

Step2: i = 1   1<<i => 010

i = 1

Part1: $X_{ra} = 0 \wedge \cancel{2} \wedge \cancel{2} \wedge 3 \Rightarrow \boxed{3}$

Part2: $X_{rb} = 0 \wedge 1 \wedge \cancel{5} \wedge \cancel{5} \wedge \cancel{4} \wedge \cancel{4} \Rightarrow \boxed{1}$

i=3 => 1000

```
int Xra = 0        Xrb = 0

for( K = 0;   K < N;   K++){
        if( A[K] & (1 << i) ) == 0){
                Xrb = Xrb ^ A[K]
        }
        else{
                Xra = Xra ^ A[K];
        }
}

[ Xra, Xrb ];
```

O(N)

$\{ \underbrace{N \,\&\, (1<<i)}_{i^{th} \text{ bit} \to \text{unset}}) == 0$

else i^{th} bit → Set

$\{ X_R \,\&\, (X_R - 1) \}\wedge X_R$

T.C: O(N) + O(log N) + O(N) = O(N)

SC: O(1)

(x_k)

$N =$ 1 0 1 0 0 0

5 4 3 2 1 0

$N-1 =$ 1 0 0 1 1 1

$\wedge$

$\underbrace{N \,\&\, (N-1)}_{(x_k \,\&\, x_{k-1})} =$ 1 0 0 0 0 0

0 0 1 0 0 0

5 4 3 2 1 0

$1 < i < 3$

$i \geq 3$

1 0 0 0

$(x_k) \wedge (x_k \,\&\, x_{k-1}) \Rightarrow O(1)$

$N \,\&\, \sim(N-1)$

$N =$ 1 0 1 0 0 0

$N-1 =$ 1 0 0 1 1 1

$\sim(N-1) =$ 0 1 1 0 0 0

0 0 1 0 0 0

$N =$ 0 0 0 0 1 0 1 0 0 0

$N-1 =$ 0 0 0 0 1 0 0 1 1 1

$\sim(N-1) =$ 1 1 1 1 0 1 1 0 0 0

N & (N(N-1))

$N = 101000$
$N-1 = 100111$

Question: Matrix Multiplication

A : [ ] $R1 \times C1$

rows cols

B : [ ] $R2 \times C2$

$A \times B = C$ $\boxed{R1 \times C2}$

#cols of A = #rows of B

$A[1][0] \times B[0][2]$
+
$A[1][1] + B[1][2]$
$A[1][2] + B[2][2]$
$A[1][3] + B[3][2]$

$$\begin{array}{c|cc} & 0 & 1 \\ \hline 0 & 4 & 5 \\ 1 & 6 & 3 \\ 2 & 1 & 2 \end{array} \quad \begin{array}{c|cccc} & 0 & 1 & 2 & 3 \\ \hline 0 & -1 & 2 & 3 & 1 \\ 1 & 2 & 3 & -1 & 0 \end{array} = \begin{array}{c|cccc} & 0 & 1 & 2 & 3 \\ \hline 0 & 8 & 23 & 7 & \_ \\ 1 & \_ & \_ & 15 & \_ \\ 2 & \_ & \_ & \_ & \_ \end{array}$$

3×2 (C_1) $\qquad 2 \times 4$ (R_2) $\qquad$ C: 3×4

C[1][2]

row col

C[0][0] = $4 \times (-1) + 5(2) =$

C[0][1] = $4(2) + 5(3) =$

C[0][2] = $4(3) + 5(-1) =$

C_1 or R_2

C[1][2] = 6(3) + 3(-1) = 15

C[i][j] = Multiply i^th row with j^th column

```
val=0;
for( k = 0;   k < C1(R2);   k++){      // C1 first
    val = val + A[i][k] * B[k][j]
}
```

$\boxed{C[i][j] = val;}$

```
for (i=0;  i< R1 ;  i++){
    for (j=0;   j < C2 ;  j++){
        val=0;
        for( k = 0;   k < C1(R2);   k++){      // C1 first
            val = val + A[i][k] * B[k][j]
        }
        C[i][j] = val;
    }
}
```

R1   C2   C1

$[R_1 \times C_2] \times C_1$

$\boxed{T \cdot C : O(R_1 \times C_2 \times C_1)}$

$\boxed{N \times N \Rightarrow O(N^3)}$

Assignments / Homework

Find max sum using B elements: O(1) space

```
// Sum of first B elements
LeftSum = sum[0… B-1]
L = B-1;
RightSum = 0;
R = n-1;
ans = max(ans, LeftSum + RightSum);

while(L >= 0){
      LeftSum -= A[L];
      L--;
      RightSum+= A[R];
      R--;
      ans = max(ans, LeftSum + RightSum);
}
return ans;
```